

An Energy-Efficient Ahead-Pipelined TAGE Branch Predictor for CBP-NG

Nhat Dang

Department of Electrical and Computer Engineering
North Carolina State University
Raleigh, NC, USA
nmdang@ncsu.edu

Eric Rotenberg

Department of Electrical and Computer Engineering
North Carolina State University
Raleigh, NC, USA
ericro@ncsu.edu

Abstract—Modern TAGE-SC-L [1] branch predictors provide high accuracy, but their large number of history tables and complex prediction and update logic introduce significant latency, wiring, and energy overheads. This work proposes a TAGE design optimized for accuracy, latency, and energy-efficiency. For latency, our contributions include techniques such as ahead-pipelining, duplication logic for fanout optimization, and wiring-cost reduction through strategic SRAM placement. We also explore energy-saving techniques, such as selective access gating for upper TAGE tables with longer histories, avoiding unnecessary reads when shorter-history components provide sufficient confidence. Our submission achieves a VFS score of 0.9742, higher than the VFS scores of the CBP-NG provided gshareN_ahead (0.9674) and TAGE (0.8728).

Index Terms—branch prediction

I. INTRODUCTION

For CBP-NG, branch prediction design must balance three competing objectives: prediction throughput, misprediction penalty, and energy efficiency. Optimizing only one dimension is insufficient. An aggressive predictor may improve accuracy, but it can also increase access latency, wiring complexity, and energy consumption. Conversely, a simpler predictor may reduce energy, but lose performance due to lower prediction throughput or a higher misprediction rate. This tradeoff becomes especially important in modern wide-fetch processors, where the front-end must predict multiple branches per cycle while keeping the prediction latency low enough to avoid fetch bubbles.

For our design, we use TAGE [2]–[4] because it is one of the state-of-the-art branch prediction algorithms for high-performance processors. TAGE achieves high accuracy by using multiple tagged tables indexed with different global-history lengths. Longer-history tables are useful for branches whose behavior depends on longer control-flow context, while shorter-history tables provide robust predictions for simpler branches. However, this accuracy comes at a hardware cost. A conventional TAGE predictor requires parallel access to multiple history tables, tag comparisons across many candidate providers, and provider/alternate-provider selection logic. When extended to support multiple branch predictions per fetch block, the cost increases further because each table may require multiple reads or more complex banking and alignment logic. For prediction throughput, we decide to use one table

for each history length instead of multiple tables for each history length. This means that consecutive branches will be guaranteed to be predicted by different history lengths. This design choice is motivated by the limited number of tables we can afford to use without exploding the energy cost and also to capture the benefit of longer history lengths.

The key latency optimization is that expensive TAGE table accesses are performed one block ahead of the point where their results are needed. By moving table access into a cycle earlier, the design reduces pressure on the critical prediction path. Instead of reading consecutive entries to generate multiple branch predictions for each missing history pattern, we employ a secondary tag for the missing history pattern similar to prior work [5].

In addition, we optimize high-fanout signals and reduce long-distance wiring cost, making the predictor more suitable for a high-frequency implementation. These optimizations allow the predictor to maintain the accuracy benefits of TAGE while reducing the delay in table indexing, tag comparison, and provider selection.

The design also aims at energy efficiency. In a full TAGE predictor, the longer-history tables are frequently accessed even when they are unlikely to provide the final prediction. These upper tables are usually larger, more expensive to access, and more dependent on long global history computation. We therefore propose selectively gating accesses to upper TAGE tables under conditions where shorter history components or the base predictor are likely to be sufficient. This reduces unnecessary table reads and reduces dynamic energy without completely removing the benefit of long history prediction.

II. DESIGN OVERVIEW

We target a single predictor with less-than-one-cycle normalized prediction latency, instead of using a two-predictor model (one that is fast and one that is slower but more accurate). This design choice avoids some complexity of the two-predictor model, such as short mispredictions due to disagreement between the two predictors and complex placement if both predictors require multiple tables. To achieve an IPC_{cbp} close to 8, which is that of the hypothetical reference design in the VFS equation, our design predicts

up to 4 branches per cycle. In more depth, the current fetch block is terminated under these conditions: (1) a taken branch, (2) a misprediction, (3) the end of a 256-instruction region, and (4) the 4th branch in the fetch block. The base predictor provides an initial prediction bundle for all branch positions, while the tagged tables override these predictions when their tags match. Our design uses geometric history lengths with the maximum history length of 100. A higher history length might reduce misprediction rate but it will also increase the latency of index generation during prediction. The base predictor is also ahead-pipelined to reduce critical path latency. It is indexed by the ahead PC, and it has multiple banks to capture multiple missing history patterns. Alternatively, we can reduce the capacity of the base predictor to meet the timing requirement, but this will increase the misprediction rate for workloads with high branch density.

Each TAGE table entry stores a prediction bit, a 2-bit hysteresis counter, a usefulness bit, and a tag. The stored tag has 11 bits, split conceptually into a 2-bit branch position field and a 9-bit hashed tag. The branch-position field allows a single block prediction to distinguish among the possible branch ranks within the block. In addition, when ahead-pipelining is enabled, each entry stores a secondary tag of 6 bits. This secondary tag is derived from missing history pattern (PC) information and is used to disambiguate entries when the TAGE lookup is performed before all history information is fully resolved. In [5], the secondary tag is generated by hashing all skipped-branch predicted targets. In our design, we generate the secondary tag using only the predicted target lower bits of the final branch in the fetch bundle. Despite using this simpler secondary tag generation method, our design achieves MPKI within 0.08 of the non-ahead-pipelined design under the same configuration, where no secondary tag is used and the main tag is 14 bits. Compared to the hash of skipped-branch predicted targets [5], our secondary tag method reduces MPKI by 0.14 MPKI and EPI_{cbp} by 1%.

During prediction, the base predictor first generates predictions for all possible branch positions in the current fetch block. TAGE computes both index and hashed tag for each tagged table using the current instruction address and the folded history registers. These table accesses are pipelined one block ahead: the predictor consumes the previously launched reads while simultaneously launching the reads for the next prediction. A TAGE entry is considered a valid match only if its hashed tag, branch position field, and secondary tag all match the current lookup context. For each possible branch position, the predictor forms a match vector across the tagged tables. The longest matching table provides the primary prediction, while the second-longest matching table provides an alternate prediction. If no tagged table matches, the predictor falls back to the base prediction. Thus, the final prediction for each branch position is its longest matching TAGE prediction when available, otherwise the corresponding base predictor output. The alternate prediction is retained for update decisions, especially for useful bit management.

On update, the predictor records the actual outcomes of

the conditional branches in the block and updates the base predictor for each branch position. For the TAGE component, it identifies the primary providers used during prediction and compares their predictions against the resolved branch outcomes. A given provider's prediction bit is changed, only if its hysteresis is weak and its prediction differs from the resolved outcome. Provider hysteresis counters are updated to strengthen or weaken confidence according to correctness. A given provider's useful bit is updated according to correctness, only when the primary and alternate predictions differ. When a misprediction occurs, the predictor attempts to allocate new entries in longer-history tables beyond the provider that matched the mispredicted branch. Candidate entries are selected from tables whose usefulness bit is clear and whose hysteresis state makes them replaceable. (If no allocation candidate is available, usefulness bits in tables after the provider table are cleared to make future allocations possible.) A newly allocated entry receives the current branch position tag, hashed tag, secondary tag, actual branch direction, initialized hysteresis and useful bit. Finally, the predictor advances its history state. The folded history is updated using a compact value hashed from the block PC, the final branch's position and direction, and the next block PC.

III. DETAILED PREDICTOR OPERATION

Figure 1 shows our TAGE design including the base predictor and the TAGE tables.

A. Base predictor

The base predictor is a 2-bit bimodal predictor split across two structures: the prediction-bit SRAM and the hysteresis-bit SRAM. All prediction and update registers are placed near the prediction-bit SRAM to reduce the wiring cost. During prediction, the base predictor does not read the SRAM and use it immediately in the same cycle. Instead, it launches the next prediction-bit read using the current block PC, while consuming the previously read result. The base predictor reads out 4 predictions (for up to 4 branches in the fetch block) from each of 8 banks using the current fetch block PC. We use missing path history to select the bank for the final predictions. The base predictor does not affect the critical path since at prediction time, it only needs to do the selection logic.

The update behavior is like a standard 2-bit bimodal counter. If the prediction was correct, the predictor makes the entry confident. If the prediction was wrong and the entry was strong, it only weakens the entry. If the prediction was wrong and the entry was already weak, it flips the prediction direction to the actual branch outcome. This gives the predictor some resistance against changing direction after a single noisy branch outcome.

B. TAGE

In the cycle before prediction, the TAGE prediction path first generates an index and a hashed tag for each tagged table using the PC and folded history. The predictor uses the index to read out the main tag, prediction bit, secondary tag, hysteresis,

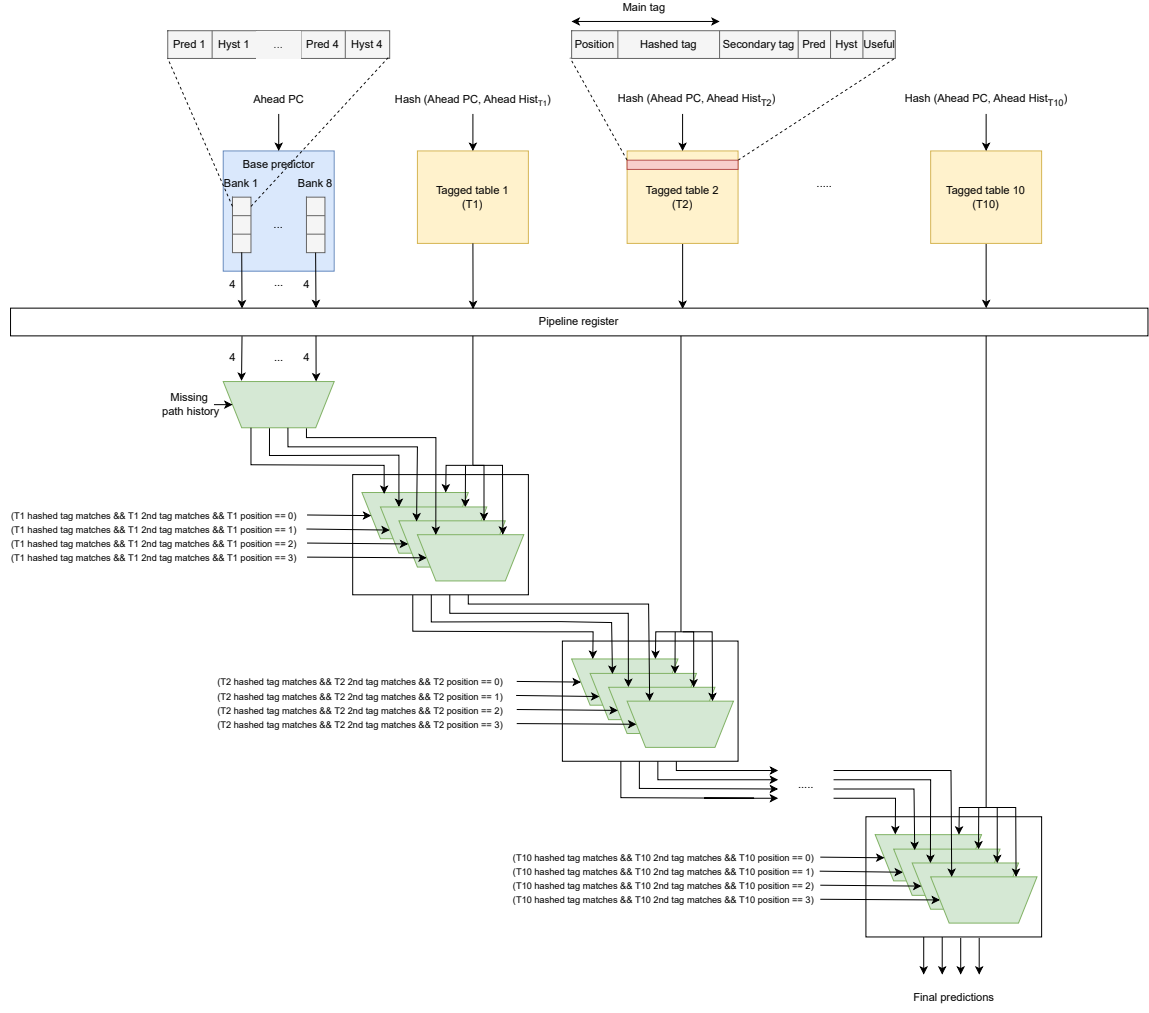


Fig. 1. TAGE design diagram.

and useful bit from respective SRAMs (thus, five SRAMs per tagged table). The critical path is dominated by the main tag read since the main tag SRAM is larger than the others. This motivates placing all logic for the main tag comparison near the main tag SRAM, to reduce the wiring cost. All read data are stored in registers to be used in the next cycle. In the prediction cycle, the predictor uses stored information in registers to do the tag comparison. This includes the hashed tag comparison, secondary tag comparison, and branch rank (position) comparison. All of these computations are done in parallel.

Updates of provider tables' prediction bits, hysteresis counters, and useful bits were explained at the end of Section II. On a misprediction, up to 2 new entries (if available) are allocated. In our design, 2-new-entries allocation gives a 0.5% reduction in misprediction rate compared to 1-new-entry allocation. Allocation chooses later tables with low usefulness and a weak enough state, then writes new main tag, secondary tag, prediction bit, hysteresis, and useful bit values. Correct updates consume relatively little energy because they only update

hysteresis counters and useful bits. Misprediction updates are heavier because they may additionally allocate new entries and write several tables.

C. Timing optimization

Some signals in our design have very high fanout, which causes significant delay. A prominent example is a signal that gates accessing the prediction tables immediately following a misprediction recovery (due to the nature of ahead-pipelining, the tables should not be accessed if the offending branch is in the middle of a block and was mispredicted as taken when actually not-taken). This signal also gates updating the folded history which is a timing-critical path. We address the high fanout, hence long delay, of this signal, by duplicating the logic that generates it.

D. Energy optimization

For energy optimization, we notice that there are several workloads which have high branch density but low branch misprediction rate even with a bimodal predictor or a small

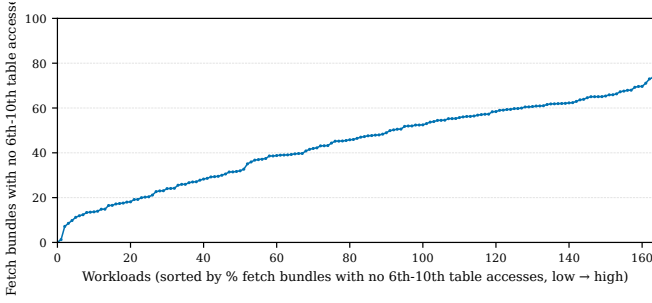


Fig. 2. Percentage of fetch bundles with no 6th-10th tagged table accesses across workloads.

gshare predictor. Also, Figure 2 shows that close to half of the workloads have over 50% of fetch bundles with no accesses to tagged tables 6 through 10. This allows us the opportunity to gate access to upper TAGE tables with longer histories since shorter history tables are likely to provide correct predictions. Our design implements selective access gating for upper TAGE tables with longer histories. We use a simple condition that if we don't see a misprediction within N_1 blocks and if the number of mispredictions $\times N_2$ is less than the number of retired branches, we gate access to upper TAGE tables with longer histories. We pick these thresholds to simplify the hardware implementation. In our design, $N_1 = 512$ and $N_2 = 1024$. Decreasing N_1 and N_2 reduces EPI_{cbp} but significantly increases CPI_{cbp} , which lowers the overall VFS score. This simple condition is effective enough that we're able to reduce the energy cost of certain workloads, such as `compress_44`, by 25% without affecting the accuracy. However, the overall VFS score does not change significantly since, for other workloads that do not benefit from this optimization, there is additional energy cost due to maintaining the counter and the gating logic for this optimization. There's also an increase in mispredictions, which lowers the overall VFS score with this optimization. The overall VFS score with this optimization is 0.9741. We commented-out this optimization for the final CBP-NG submission.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

We compare our predictor with the provided TAGE and gshareN_ahead designs across the CBP-NG workload suite using the competition modeling framework. Figures 3–6 summarize prediction throughput (IPC_{cbp}), branch MPKI, long misprediction penalty in lost cycles per correct-path instruction (CPI_{cbp}), and dynamic energy per correct-path instruction (EPI_{cbp}).

Figure 3 shows that our design achieves prediction throughput close to that of gshareN_ahead. Even though gshareN_ahead can predict up to 7 branches per cycle, versus 4 in our design, our design is able to achieve similar prediction throughput for two reasons: (1) the fetch bundle is often terminated for other reasons, such as a taken branch, a misprediction, or the end of a region, and (2) our TAGE-based design has less frequent mispredictions. The provided TAGE has significantly lower prediction throughput since it

TABLE I
AVERAGE IPC_{cbp} , CPI_{cbp} , EPI_{cbp} , AND VFS ACROSS THE CBP-NG WORKLOAD SUITE.

Predictor	IPC_{cbp}	CPI_{cbp}	EPI_{cbp}	VFS
Our TAGE design	8.617264	0.051773	1068.392857	0.974177
gshareN_ahead	9.172020	0.066203	243.916667	0.967410
Provided TAGE	5.768926	0.054894	1265.452381	0.872846

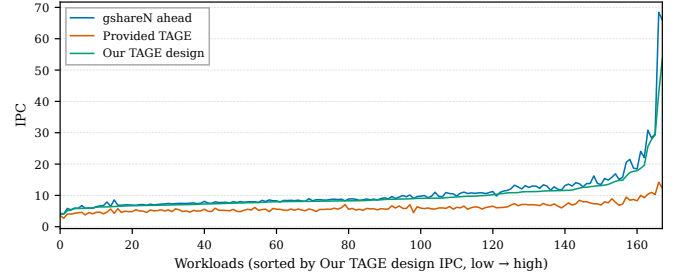


Fig. 3. Prediction throughput in instructions per cycle (IPC_{cbp}).

terminates at a 16-instruction line boundary. The provided TAGE also uses a P1/P2 approach, with P1 being a gshare predictor and P2 being a TAGE predictor. There are several workloads where P1 and P2 disagreement significantly reduces the prediction throughput.

Our design achieves similar MPKI as the provided TAGE (Figure 4), even with ahead context and smaller tagged tables storage. Although average MPKI of the provided TAGE is slightly lower, its CPI_{cbp} is slightly higher (Figure 5) owing to one extra cycle in the per-long-misprediction penalty. Our design achieves lower EPI_{cbp} than the provided TAGE (Figure 6), due to a smaller tag SRAM size. Moreover, even though our design uses ahead context, the number of reads is the same because of the secondary tag strategy. GshareN_ahead has significantly lower EPI_{cbp} because it reads far fewer bits to make a prediction.

Our design achieves an average IPC_{cbp} of 8.62, CPI_{cbp} of 0.052, and EPI_{cbp} of 1,068. This gives a VFS score of 0.9742, whereas the VFS scores of the provided gshareN_ahead and TAGE are 0.9674 and 0.8728 (Table I).

Table II shows the VFS score of our TAGE design after each timing optimization. Ahead-pipelining and strategic logic placement provide the largest improvement, by reducing the predictor latency from 1.7 cycles to 0.9767 cycles. The lower latency directly improves IPC_{cbp} from 4.47 to 8.5835, which leads to an increase in VFS score from 0.7813 to 0.9718. Simplifying the secondary tag improves the VFS score from 0.9718 to 0.9742 due to a reduction in MPKI, which leads to an increase in IPC_{cbp} from 8.5835 to 8.6172. When the predictor latency is below one cycle, further latency reduction does not improve IPC_{cbp} . However, lower latency provides opportunities to add components, such as a statistical corrector predictor, or increase the size of tagged tables.

Table III shows the storage breakdown of our TAGE predictor.

TABLE II
TAGE DESIGN LATENCY AFTER EACH TIMING OPTIMIZATION.

Configuration	Latency (cycles)	IPC_{cbp}	VFS score
TAGE (without optimization)	1.7	4.47	0.7813
Ahead-pipelined	1.1	4.47	0.7731
Ahead-pipelined and strategic logic placement	0.9767	8.5835	0.9718
Ahead-pipelined, strategic logic placement, and simplified secondary tag	0.926	8.6172	0.9742
Ahead-pipelined, strategic logic placement, simplified secondary tag, and fanout optimization	0.86	8.6172	0.9742

TABLE III
STORAGE BREAKDOWN OF OUR TAGE PREDICTOR

Component	Entry structure	Size
Base predictor	Prediction bit: 2^{12} 1-bit entries per bank, hysteresis bit: 2^{12} 1-bit entries per bank, 8 banks, 4 prediction lanes. Total storage: $2^{12} * 8 * 4 * 2$	262,144 bits
	Ahead prediction pipeline register: (8 banks) * 4 prediction lanes	32 bits
TAGE	Tagged entry: Tag: 11 bits, secondary tag: 6 bits, prediction: 1 bit, hysteresis: 2 bit, usefulness: 1 bit; 10 tagged tables with 1024 entries each. Total storage: $1024 * (11 + 6 + 1 + 2 + 1) * 10$ tables	215,040 bits
	Global history register: 100 bits, folded history register for index per table: 10 bits, folded history register for hashed tag per table: 9 bits. Total storage: $100 + (10 + 9) * 10$ tables	290 bits
	Index computation register: 10 bits, Hashed tag computation register: 9 bits, hashed tag ahead pipeline register: 11 bits, secondary tag ahead pipeline register: 6 bits, prediction ahead pipeline register: 1 bit, hysteresis ahead pipeline register: 2 bits, usefulness ahead pipeline register: 1 bit. Total storage: $(10 + 9 + 11 + 6 + 1 + 2 + 1) * 10$ tables	400 bits
	Power gating control register: 1 bit power gating control, 18 bits for misprediction counter, 11 bits for consecutive correct prediction counter, 20 bits for conditional branch counter.	50 bits
TOTAL		58.34 KB

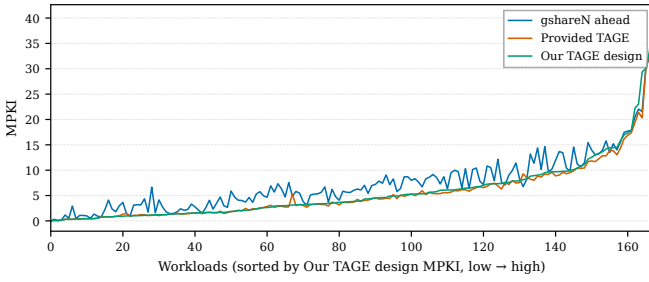


Fig. 4. Branch MPKI.

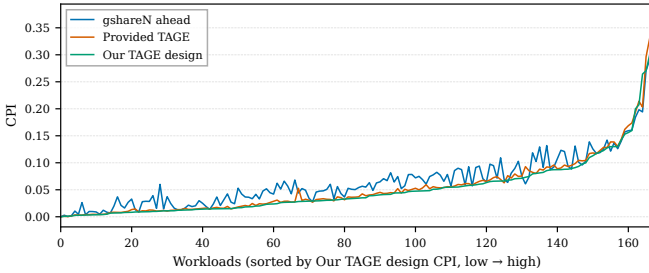


Fig. 5. Long misprediction penalty in lost cycles per correct-path instruction (CPI_{cbp}).

V. CONCLUSION

This work presents an ahead-pipelined TAGE branch predictor that balances prediction accuracy, latency, and dynamic energy. The experimental results show that the proposed design achieves prediction throughput close to the gshareN_ahead baseline while maintaining branch MPKI close to the provided TAGE baseline. Compared with the provided TAGE design, the ahead-pipelined predictor reduces the long misprediction penalty because it avoids the slower P1/P2 prediction structure.

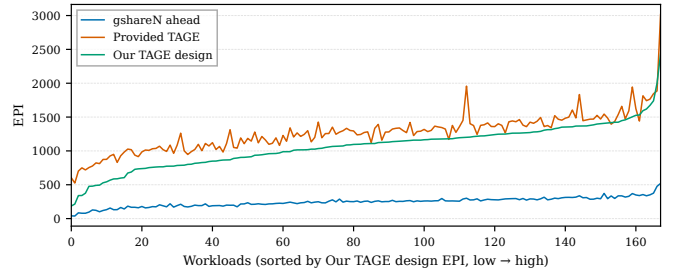


Fig. 6. Dynamic energy per correct-path instruction (EPI_{cbp}).

It also lowers energy relative to the provided TAGE design through smaller tagged-table storage and selective gating of upper TAGE tables. Although the gating policy provides clear savings on some workloads, its effect on the overall VFS score is limited by workloads that do not benefit and by the overhead of the gating counters and control logic. Overall, the design achieves a VFS score of 0.9742 with respect to the hypothetical reference design.

Future work would be to explore more adaptive confidence mechanisms for deciding when to access long-history tables or add an SC component [6] to the design to further reduce the branch misprediction rate. The current gating policy is intentionally simple, but a per-branch confidence estimator could reduce energy without adding unnecessary overhead on workloads that need long histories. Another direction is to tune the number of tagged tables, table sizes, and secondary-tag widths to better trade off accuracy against SRAM energy and tag-match delay. Finally, a more detailed physical implementation study would help quantify the benefits of the proposed fanout and placement optimizations under realistic wire-delay and banking constraints.

REFERENCES

- [1] André Seznec. 2024. TAGE: an engineering cookbook. Technical Report 9561. Inria. 1–73 pages. <https://hal.science/hal-04804900>
- [2] André Seznec. 2016. TAGE-SC-L branch predictor again. 5th Championship Branch Prediction. In Proceedings of the 5th Championship Branch Prediction.
- [3] André Seznec. 2014. TAGE-SC-L Branch Predictors. In Proceedings of the 4th Championship Branch Prediction Workshop (CBP-4).
- [4] André Seznec and Pierre Michaud. 2006. A case for (partially)-tagged geo metric history length predictors. Journal of Instruction Level Parallelism (<http://www.jilp.org/vol8>) (April 2006). <http://www.jilp.org/vol8>
- [5] Lingzhe(Chester) Cai, Aniket Deshmukh, and Yale Patt. 2025. Enabling Ahead Prediction with Practical Energy Constraints. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25). Association for Computing Machinery, New York, NY, USA, 559–571. <https://doi.org/10.1145/3695053.3730998>
- [6] Daniel A. Jiménez. 2016. Multiperspective Perceptron Predictor. In 5th JILP Work shop on Computer Architecture Competitions (CBP-5). <https://jilp.org/cbp2016/paper/DanielJimenez1.pdf>